

Atividade Prática: Código de Cadeia

Luiz Eduardo da Silva

2026-06-18

Table of contents

1	Objetivo	2
2	Introdução	2
3	Parte I — Decodificação do Código de Cadeia	3
3.1	Problema	3
3.2	Formato da Entrada	3
4	Parte II — Codificação do Código de Cadeia	3
4.1	Problema	3
4.2	Validação	3
5	Detalhamento do processo da Codificação e Decodificação de Contornos	4
5.1	Representação de Contornos	4
5.2	Sistema de Direções	4
6	Processo de Codificação	5
6.1	Passo 1 – Localização do ponto inicial	5
6.2	Passo 2 – Rastreamento do contorno	5
6.3	Passo 3 – Geração do código de cadeia	6
6.4	Passo 4 – Conversão para binário	6
6.5	Passo 5 – Compactação em hexadecimal	7
7	Processo de Decodificação	7
7.1	Passo 1 – Leitura do arquivo	8
7.2	Passo 2 – Conversão para binário	8
7.3	Passo 3 – Recuperação das direções	8
7.4	Passo 4 – Reconstrução do contorno	9
7.5	Passo 5 – Geração da imagem	9

1 Objetivo

O objetivo desta atividade é explorar técnicas de **representação de formas e contornos** em Processamento Digital de Imagens por meio da implementação de um sistema completo de:

1. **Codificação de contornos** utilizando código de cadeia oito-direcional;
2. **Decodificação de contornos** para reconstrução da imagem original.

Ao final da atividade, o estudante deverá ser capaz de converter um contorno em uma representação compacta e reconstruí-lo posteriormente.

2 Introdução

O **código de cadeia** (*chain code*) é uma técnica clássica de representação de contornos proposta por Freeman.

A ideia consiste em armazenar:

- um ponto inicial do contorno;
- a sequência de direções percorridas ao longo da borda do objeto.

Para a vizinhança oito-direcional utiliza-se a seguinte convenção:

```
3 2 1
4 x 0
5 6 7
```

e os deslocamentos:

```
typedef struct {
    int i, j;
} point;

point N[8] = {
    { 0, 1},
    {-1, 1},
    {-1, 0},
    {-1, -1},
    { 0, -1},
    { 1, -1},
    { 1, 0},
```

```
{ 1, 1}  
};
```

3 Parte I — Decodificação do Código de Cadeia

3.1 Problema

Desenvolva um programa capaz de reconstruir um contorno a partir de sua representação codificada.

3.2 Formato da Entrada

```
<nl> <nc>  
<linha_inicial> <coluna_inicial>  
<n_direcoes>  
<cadeia_hexadecimal>
```

4 Parte II — Codificação do Código de Cadeia

4.1 Problema

Desenvolva um programa capaz de gerar o código de cadeia de um contorno presente em uma imagem binária.

4.2 Validação



5 Detalhamento do processo da Codificação e Decodificação de Contornos

5.1 Representação de Contornos

Em muitas aplicações de Processamento Digital de Imagens, não é necessário armazenar todos os pixels de um objeto. Frequentemente, apenas seu contorno é suficiente para representar sua forma.

O **código de cadeia** (*chain code*) é uma técnica de representação de contornos proposta por Freeman, na qual um objeto é descrito por:

- um ponto inicial do contorno;
- uma sequência de direções que indicam como percorrer a borda do objeto.

Dessa forma, em vez de armazenar todos os pixels da imagem, armazena-se apenas a trajetória percorrida ao longo do contorno.

5.2 Sistema de Direções

Nesta atividade será utilizado o código de cadeia de **8 direções**.

```
3 2 1
4 x 0
5 6 7
```

Cada direção corresponde a um deslocamento em relação ao pixel atual.

Código	Deslocamento	Direção
0	(0,+1)	Leste
1	(-1,+1)	Nordeste
2	(-1,0)	Norte
3	(-1,-1)	Noroeste
4	(0,-1)	Oeste
5	(+1,-1)	Sudoeste
6	(+1,0)	Sul
7	(+1,+1)	Sudeste

6 Processo de Codificação

A codificação consiste em transformar um contorno presente em uma imagem em uma sequência compacta de direções.

6.1 Passo 1 – Localização do ponto inicial

Inicialmente deve-se localizar um pixel pertencente ao contorno.

Uma estratégia simples consiste em percorrer a imagem da esquerda para a direita e de cima para baixo até encontrar o primeiro pixel com valor 1.

Por exemplo:

```
0 0 0 0
0 1 1 0
0 1 0 0
0 0 0 0
```

O ponto inicial é:

```
(1,1)
```

6.2 Passo 2 – Rastreamento do contorno

Partindo do ponto inicial, o algoritmo percorre os pixels vizinhos pertencentes ao contorno.

Para cada movimento realizado:

1. identifica-se qual direção foi utilizada;
2. registra-se seu código;
3. avança-se para o próximo pixel.

Exemplo:

```
(2,1)
↓
(3,1)
↓
(4,1)
→ (4,2)
→ (4,3)
```

Direções:

```
6 6 0 0
```

6.3 Passo 3 – Geração do código de cadeia

Após percorrer todo o contorno obtém-se uma sequência:

```
7 5 6 0 0 0 2 2 2 1 4 4 4
```

Essa sequência constitui o código de cadeia do objeto.

6.4 Passo 4 – Conversão para binário

Como existem apenas oito direções possíveis, cada direção pode ser representada por 3 bits.

Direção	Binário
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Exemplo:

```
7 5 6 0
```

torna-se

```
111 101 110 000
```

6.5 Passo 5 – Compactação em hexadecimal

Os bits produzidos são agrupados em blocos de quatro.

Exemplo:

```
1111 0111 0000
```

resultando em:

```
F 7 0
```

O arquivo final armazena:

```
dimensões da imagem  
ponto inicial  
número de direções  
cadeia hexadecimal
```

7 Processo de Decodificação

A decodificação realiza a operação inversa.

Seu objetivo é reconstruir o contorno original a partir da representação compactada.

7.1 Passo 1 – Leitura do arquivo

O programa lê:

```
7 7  
2 1  
13  
F700124648
```

7.2 Passo 2 – Conversão para binário

Cada dígito hexadecimal é convertido para 4 bits.

```
F = 1111  
7 = 0111  
0 = 0000  
...
```

Obtendo:

```
1111011100000000...
```

7.3 Passo 3 – Recuperação das direções

Os bits são agrupados em blocos de três.

```
111 101 110 000 ...
```

produzindo:

```
7 5 6 0 ...
```

7.4 Passo 4 – Reconstrução do contorno

A partir do ponto inicial:

(2,1)

cada direção gera um novo ponto.

Por exemplo:

```
Direção 7 → (3,2)
Direção 5 → (4,1)
Direção 6 → (5,1)
Direção 0 → (5,2)
```

Os pixels visitados são marcados na imagem.

7.5 Passo 5 – Geração da imagem

Após processar todas as direções, obtém-se novamente o contorno original.

A imagem reconstruída é gravada no formato PBM.